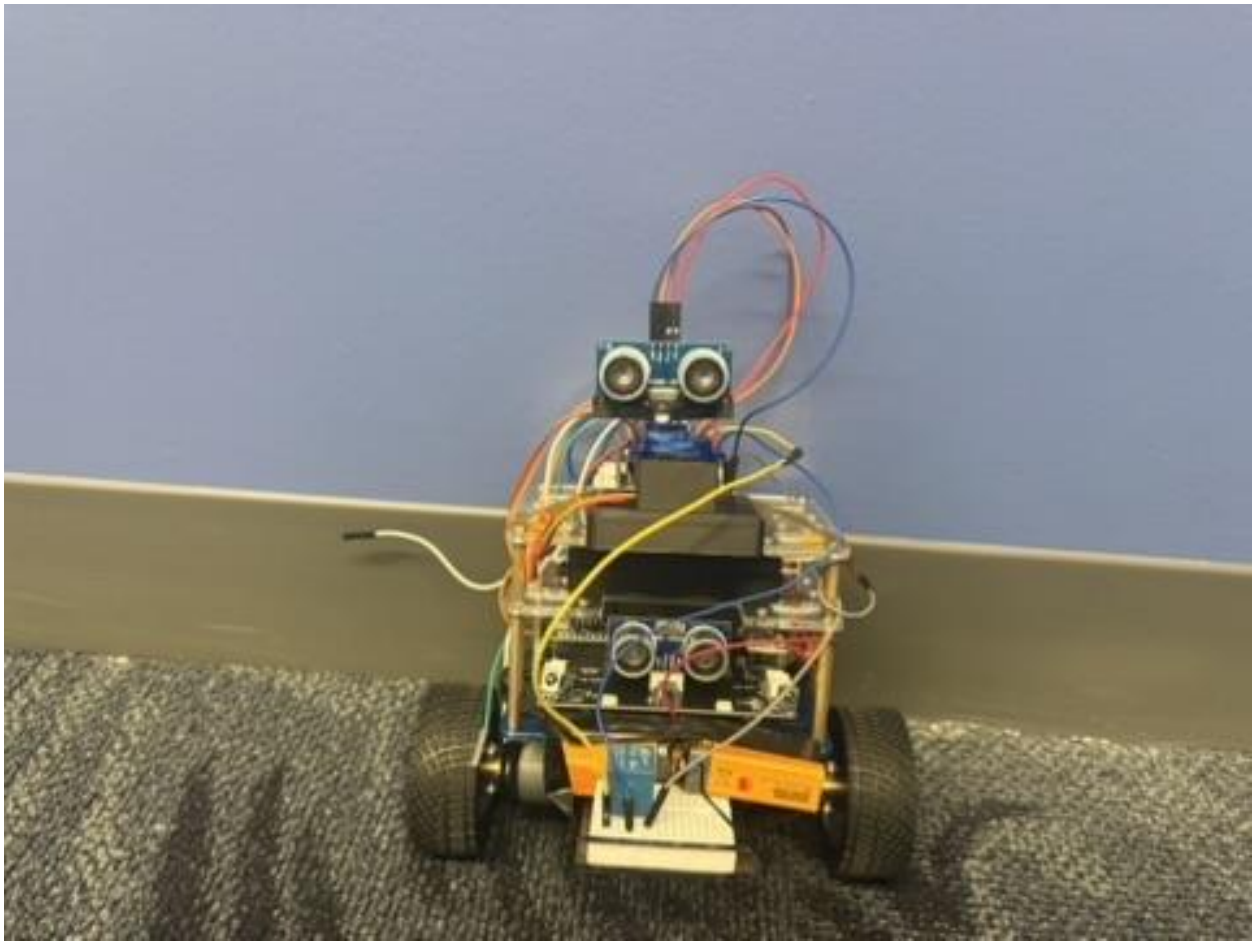
WALL-E

ME 331 Mechatronics Final Project

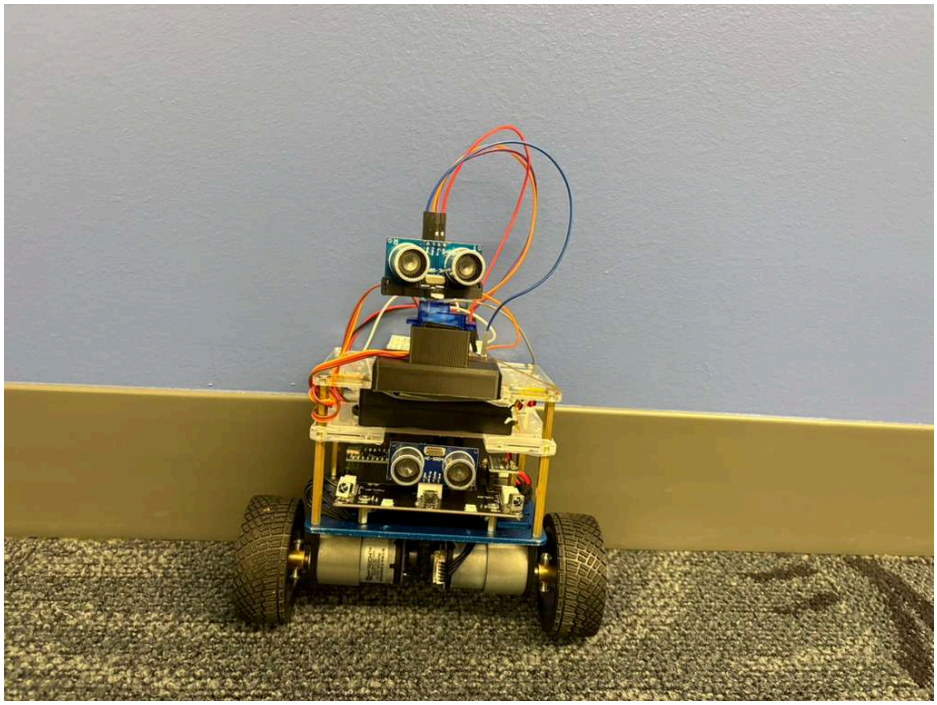
Elias Reyes & Karina Champaneri

Professor Colleen Berg

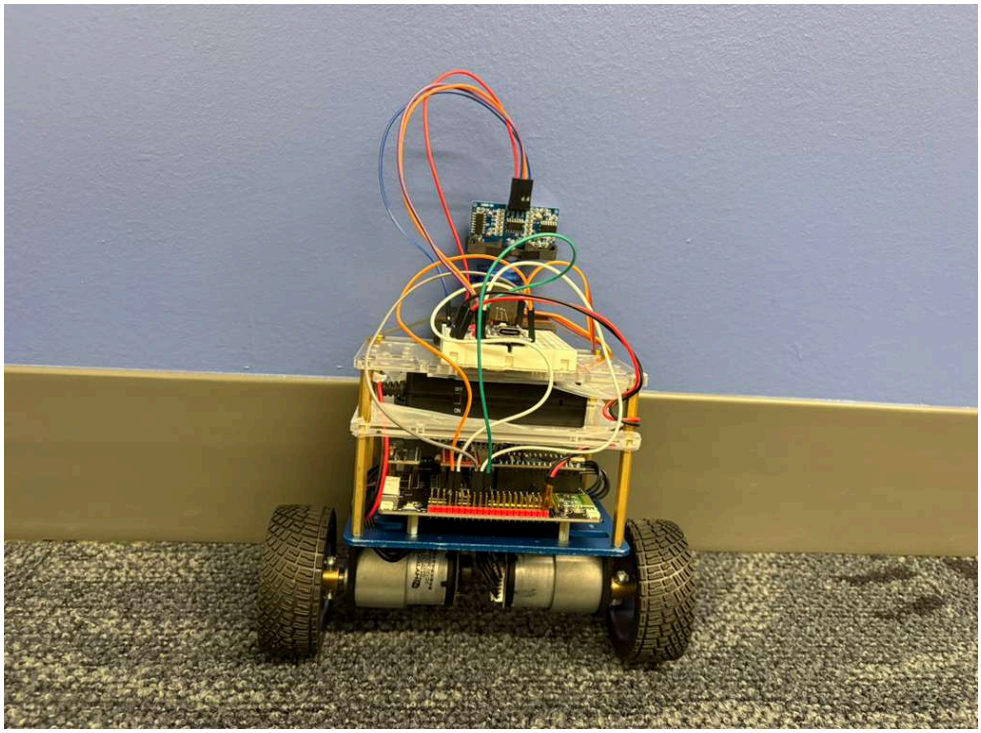


Wall-E Pictures

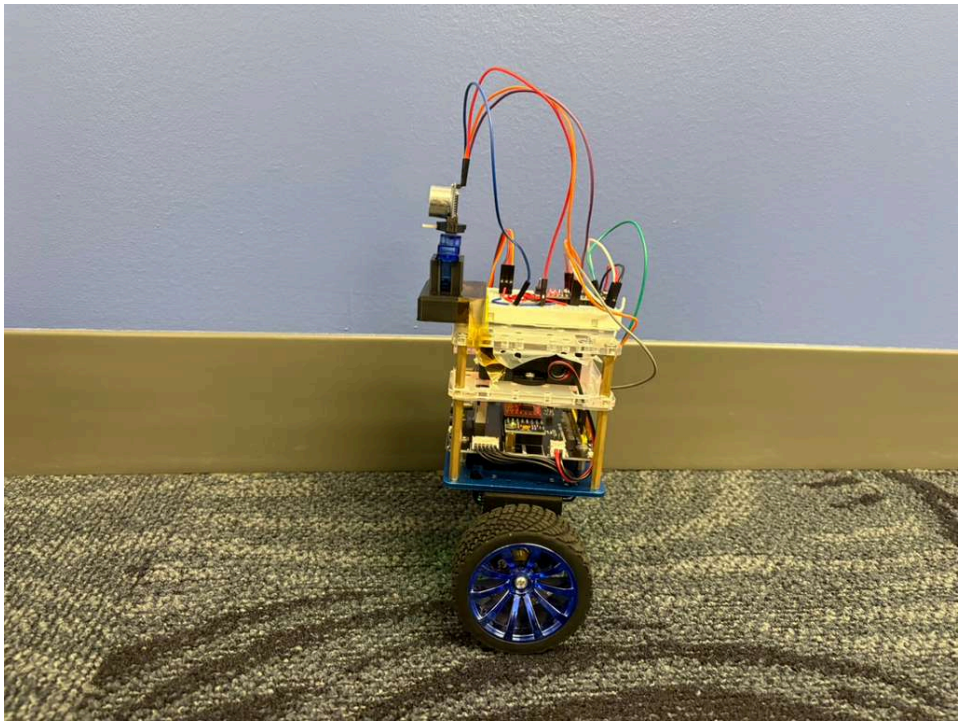
Front:



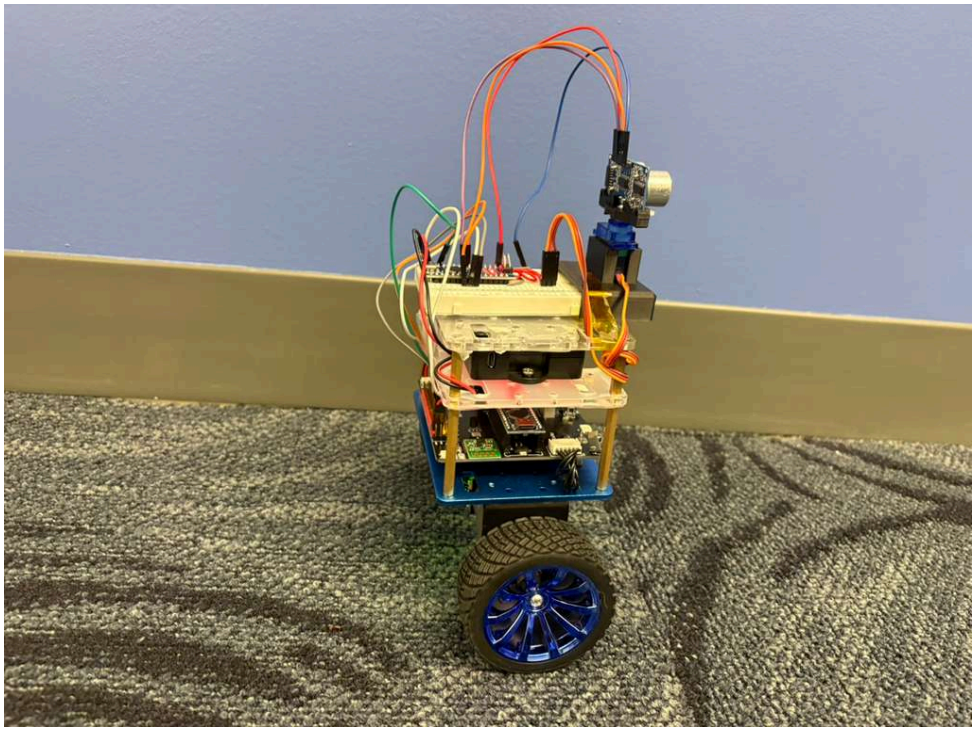
Back:



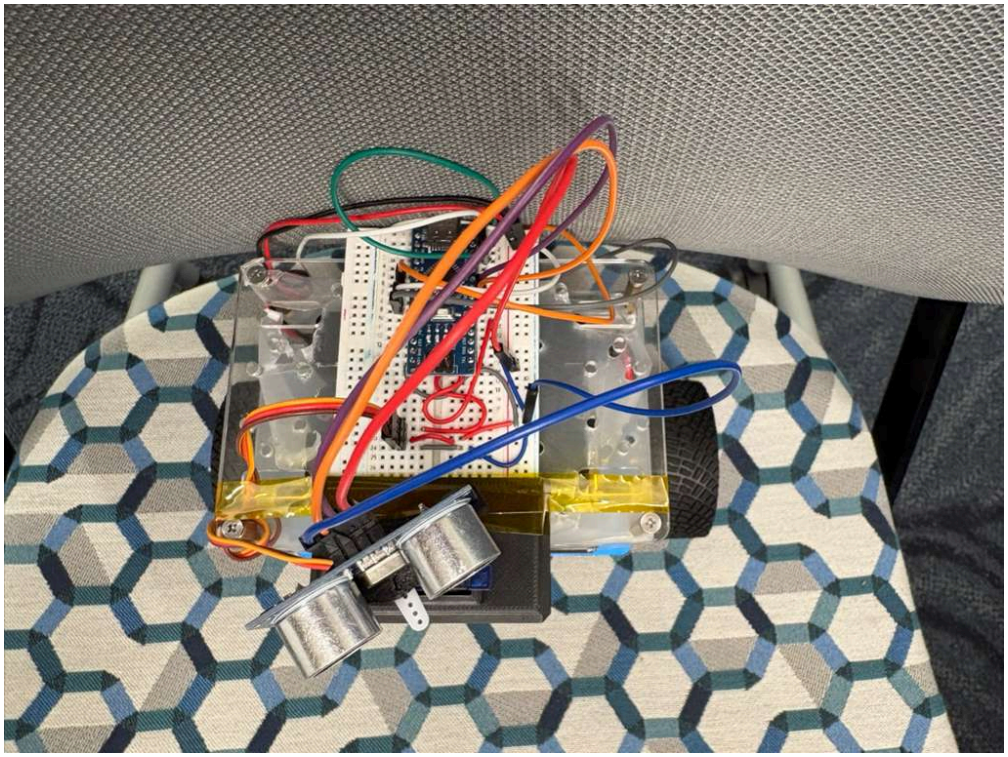
Right:



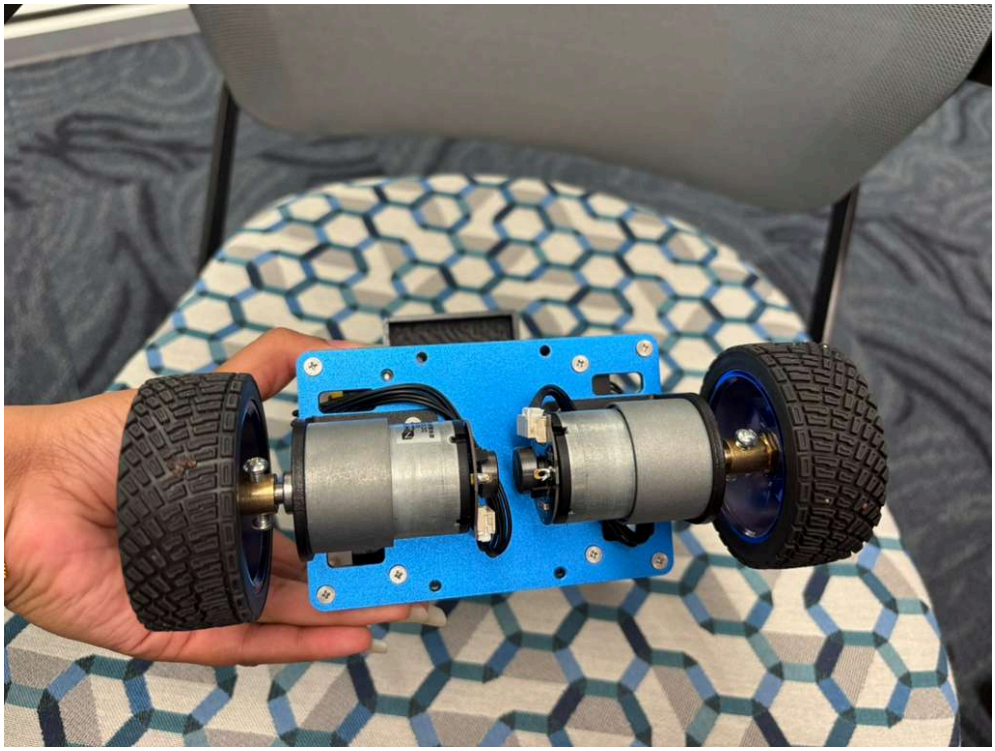
Left:



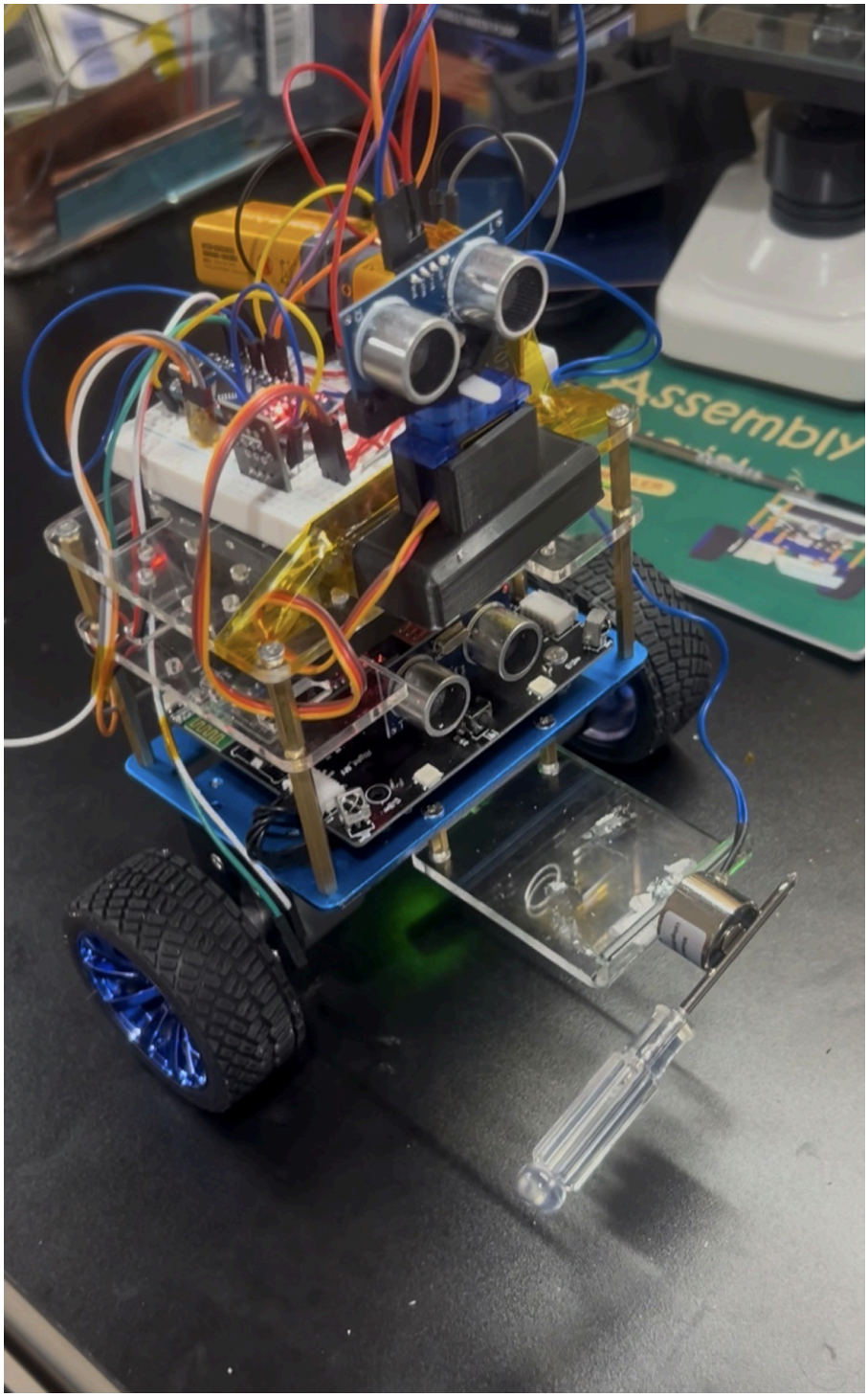
Top:



Bottom:



With electromagnet attachment:



Parts List

Part Name	Source of Part
Rover Chassis (Elegoo Tumbllr Kit)	Purchased on Amazon ($55)
Ultrasonic Sensor	Elegoo Kit
Breadboard / Wires	Elegoo Kit
Arduino Nano (Bottom)	Came with Elegoo Tumbllr Kit
Arduino Nano (Top)	Previously Owned
Magnet	Previously Owned
Electromagnet	Previously Owned
Mosfet	Previously Owned
Joystick	Elegoo Kit

References

Reference	Link	Purpose
Robot Build Tutorial	https://www.amazon.com/dp/B07QWJH77V?ref=ppx_yo2ov_dt_b_fed_asin_title	Reference guide for building the physical robot chassis
Programming the Elegoo Tumblrr	https://medium.com/@tomw3115/programming-the-elegoo-tumbller-self-balancing-robot-part-6-4826b2b9f66d	Guidance for implementing self-balancing logic and stability control.
Libraries	https://bitbucket.org/teckel12/arduino-new-ping/wiki/Home https://github.com/jrowberg/i2cdevlib	Used to provide optimized timing and distance calculation logic for the ultrasonic sensor's obstacle detection.
Schematics	https://www.elegoo.com/blogs/arduino-projects/elegoo-tumbller-self-balancing-robot-car-tutorial?srsltid=AfmBOopTw8HoXgSdW-60x2KtHIdJL8DRU4-xLoOxJMPdnCLLud33DqTU	Reference for wiring and interface pin assignments.
HW-042 MOSFET Tutorial	https://dronebotworkshop.com/transistors-mosfets/	Reference for driving and switching high power load (electromagnet)

Team Write-up

Elias:

I contributed to building the robot and ensuring the schematics and wiring were accurate and effective. I worked on tuning values to achieve optimal balance. On the programming side, I wrote, edited, and debugged code for Part 1 and developed the base code for Part 2. For Part 2, I implemented and wired the ultrasonic sensor and integrated the servo motor. For Part 3, I programmed the remote, RF communication, the electromagnet, and debugged it. I also worked on finalizing the report for this project, and added any references for parts used. My contributions focused on both system and hardware integration and developing reliable, functional code across all parts of the project.

Karina:

I contributed to building the robot and ensuring all components were properly connected. I also researched methods to achieve stable balancing and worked on tuning PID values and determining accurate encoder pulse rates to improve balance. For Part 1, I implemented and debugged the ultrasonic sensor and developed the corresponding code. For Part 2, I contributed to editing and writing the code. For Part 3, I programmed the RF transmitter, remote, and electromagnet, and wrote the code. I worked on the report as well, ensuring we had documentation for everything we used during this project. Overall, I worked on both hardware and system interface to improve the robot's capabilities and ensure there was consistent performance.

Citations

The specific examples cited below provided the foundation for our logic, though we reused and adapted these functions in multiple other areas of the program to keep the robot's movements and sensor readings consistent. This allowed us to apply the same reliable communication across the entire project.

1. **balanced_car_Challenge2**

   I used AI assistance because my robot was correctly receiving serial data values but failed to implement them consistently, likely due to hidden data corruption during transmission. Using ChatGPT, I asked, "My robot is receiving the correct serial values for angle and direction, but it isn't moving correctly." This helped my code by introducing a validation step that compares the sum of received variables against a transmitted key, called checksum. This prevents the robot from acting on non-working data and ensures it only executes movements when the data is 100% verified.

2. **slavesketch_challenge2**

   I needed to ensure that the I2C communication between the Master and Slave devices did not hang or crash if the sensor data was not yet calculated when requested. I used ChatGPT to ask, "Can you explain how a Slave device should respond to Wire.requestFrom(address, 3) if the requested data isn't calculated? Can you give me code that sends 3 bytes of '0' as a placeholder to prevent communication errors on the I2C"? This helped my code by fulfilling the Master's request for a 3-byte transmission

every time. By sending placeholder zeros, the I2C bus remains stable even when the Slave is still processing information.

3. **Challenge2REMOTE_Balanced**

   While my hardware was successfully detecting IR pulses, I needed a structured way to map those raw codes to specific robot movement commands. I used Gemini AI with the prompt, "Can you help me fix my Arduino code? My IR receiver is successfully decoding signals in the serial monitor, but the robot isn't reacting to the button presses." This helped by organizing my main loop and allowed me to easily assign remote buttons (like Power or Volume) to specific functions, making the robot's responses much more reliable and easier to debug.

4. **Challenge2_Remote_Slave**

   I used ChatGPT help to prevent distance sensor values exceeding 255 from and causing mathematical errors in my checksum calculation. I used AI to ask, "Can you help me resolve an issue where my distance sensor values are occasionally causing my checksum to fail? My distance can go above 255, but I only want to send 1 byte per variable." This helped my code by capping the distance values at a maximum of 255. This ensures that the variables never exceed the 8-bit limit of a single byte, keeping the checksum calculation accurate and preventing the Master device from rejecting valid distance data.

5. **Tom Elegoo Medium Article**

   This article was a key starting point for us. Since neither of us has taken Systems and Controls yet, we had to do our own research to understand how PID values actually work to keep a robot balanced. The step-by-step breakdown in the article helped us figure out how to tune those values, and we adapted segments of the author's code to get our robot driving steadily without tipping over. It also included detailed breakdowns on how we should program our robot to turn, as the logic is different compared to a traditional non-balancing robot.